

RAG against the machine

A Retrieval-Augmented Generation pipeline over the vLLM codebase

42 curriculum project • subject en.subject.pdf, version 2.0

Author: sloubiat

This report documents the full RAG pipeline built for this project: how the vLLM codebase is ingested and chunked, how a BM25 index is built and queried, how a local LLM (Qwen3-0.6B) generates grounded answers from retrieved context, how retrieval quality is evaluated with recall@k, and the concrete performance numbers measured on the real corpus — including the design decisions and debugging challenges behind each stage.

Corpus	vLLM 0.10.1 — 1969 indexable files, 18733 chunks
Retrieval	BM25 (bm25s), lexical, with identifier-aware tokenization
Generation	Qwen/Qwen3-0.6B (transformers), CPU-only
Evaluation	Recall@k (k=1,3,5,10), IoU ≥ 5% source matching
Indexing time	3.4s (requirement: < 5 min)
Throughput	200 questions in 0.38s (requirement: < 90s)
Recall@5 — docs	84.0% (requirement: ≥ 80%)
Recall@5 — code	70.7% (requirement: ≥ 50%)

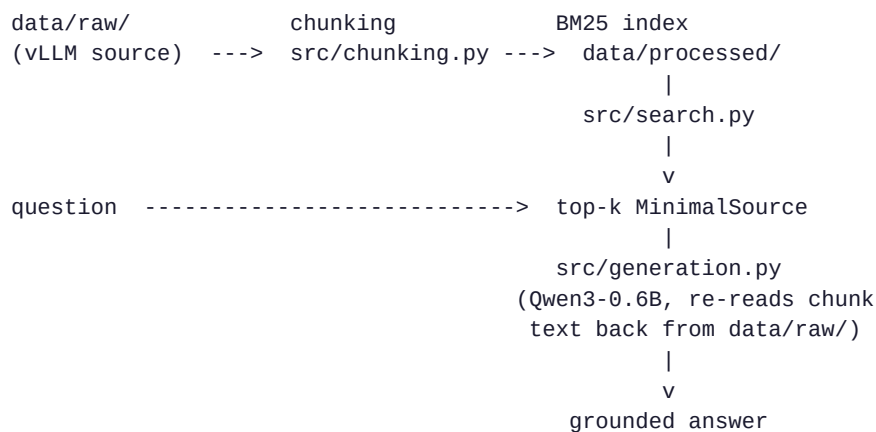
1. Description

This project implements a Retrieval-Augmented Generation (RAG) system in Python that answers natural-language questions about the [vLLM](#) codebase — a large, real-world mix of Python source and Markdown/RST documentation (3226 files total, 1969 of which are indexable). Given a question, the system retrieves the most relevant chunks of code or documentation and uses a small local LLM to generate a grounded, sourced answer.

The pipeline covers the full loop expected by the subject: ingesting and chunking the codebase, building a lexical index, retrieving the top-k relevant chunks for a query, generating an answer from that context, and evaluating retrieval quality with a `recall@k` metric — all exposed as CLI commands.

2. System architecture

The pipeline is organized as five independent stages, each backed by its own module under `src/`, sharing a common set of pydantic data contracts.



`src/evaluation.py`: `recall@k` of retrieved sources against AnsweredQuestions ground truth (own use only, never the official moulinette scorer)

Every stage is wired together by a Python Fire CLI (`src/cli.py` + `src/__main__.py`), invoked as `uv run python -m src <command>`.

<code>src/models.py</code>	Pydantic data contracts exchanged between every stage (MinimalSource, RagDataset, StudentSearchResults, ...).
<code>src/chunking.py</code>	Splits raw files into bounded-size chunks (AST-based for Python, paragraph-based for Markdown).
<code>src/indexer.py</code>	Walks <code>data/raw/</code> , chunks every file, builds and persists the BM25 index under <code>data/processed/</code> .
<code>src/tokenization.py</code>	Identifier-aware text expansion (snake_case/camelCase splitting) applied before BM25 tokenization.
<code>src/search.py</code>	Loads the index and retrieves the top-k chunks for a query.
<code>src/generation.py</code>	Turns retrieved chunks into a grounded answer with Qwen3-0.6B.
<code>src/evaluation.py</code>	Computes <code>recall@k</code> against ground-truth sources, for iteration purposes only.

3. Data models

All data exchanged between stages is validated with pydantic models defined in `src/models.py`, matching the schema fixed by the subject:

- **MinimalSource** — a file path plus a character range (`first_character_index`, `last_character_index`).
- **UnansweredQuestion / AnsweredQuestion** — a question, optionally with ground-truth sources and an answer.
- **RagDataset** — a list of Answered/UnansweredQuestion, used for every dataset file on disk.
- **MinimalSearchResults / MinimalAnswer** — retrieved sources for one question, optionally with a generated answer.
- **StudentSearchResults / StudentSearchResultsAndAnswer** — the batch output formats written by `search_dataset` and `answer_dataset`.

4. Chunking strategy

Two chunkers are implemented in `src/chunking.py`, both bounded by a configurable `max_chunk_size` (2000 characters by default, hard-enforced):

Python — `chunk_python_source`

Parses the file with the built-in `ast` module. Each top-level function or class definition (decorators included) becomes its own chunk; other top-level statements are grouped together. Any chunk exceeding `max_chunk_size` is hard-split into smaller pieces. If the file has invalid syntax, the chunker falls back gracefully to a plain hard-split of the raw text instead of crashing — the `ast` module exposes `node.lineno` and `node.end_lineno` for every top-level node, which is all that's needed to carve out exact character spans.

Markdown / text — `chunk_markdown_source`

Splits on blank-line separated paragraphs. A Markdown header (`#` through `#####`) always forces a new chunk boundary, keeping documentation sections coherent. Oversized paragraphs are hard-split the same way as the Python chunker.

This structure-aware approach keeps semantically meaningful units (a whole function, a whole doc section) together whenever they fit, instead of chunking on a fixed character/token window that could cut a function signature away from its body. On the real corpus this produces 18733 chunks from 1969 files, all verified to respect the `max_chunk_size` bound (0 violations found by walking every chunk's stored character range).

5. Retrieval method

Retrieval uses **BM25** (via the `bm25s` library) rather than TF-IDF: BM25's term-frequency saturation and length normalization handle the vLLM corpus's mix of short docstrings, long source files and prose documentation better than raw TF-IDF would, without needing a trained embedding model or a GPU.

`src/indexer.py::build_index` tokenizes every chunk and builds the index;
`src/search.py::search` tokenizes the query the same way and retrieves the top-k chunks by BM25 score, returning `MinimalSource` objects reconstructed from the persisted index metadata. The index is saved to `data/processed/` so it never needs to be rebuilt between runs.

Identifier-aware tokenization (a key improvement)

`bm25s` tokenizes on plain word boundaries, so a code identifier like `get_kv_cache_size` or `KVCacheSize` stays a single opaque token — it never matches a natural-language query such as “What is a KV cache?” (tokens `kv`, `cache`). `src/tokenization.py::expand_identifiers` fixes this by appending space-split `snake_case/camelCase` subwords for every multi-part identifier — `get_kv_cache_size` is also indexed as `get kv cache size` — on top of the original text, applied identically to both the corpus (indexing) and the query (search). Because it is purely additive, exact-identifier matching is preserved while natural-language matching is added on top.

This single change raised code-question recall@5 from 50.5% to 70.7% (+20.2 points) with no cost to documentation recall (see §7, Performance analysis).

6. Answer generation

Answers are generated locally with **Qwen/Qwen3-0.6B** via transformers (the mandatory default model), run on CPU — the model is small enough that no GPU is needed.

- **Grounding.** MinimalSource only stores a file path and a character range, not the chunk's text. The actual text is re-read from data/raw/ at generation time (src/generation.py::_read_source_text), keeping the index small and content single-sourced.
- **Prompting.** The retrieved chunks are assembled into a context block and the model is instructed to answer using only that context and to say so explicitly if the answer isn't in it (src/generation.py::_build_prompt).
- **Token budget.** MAX_CONTEXT_CHARS caps how much retrieved text is fed into the prompt, truncating rather than exceeding the model's context window.
- **Suppressing reasoning noise.** Qwen3 defaults to emitting a <think>...</think> block before its answer. Passing enable_thinking=False to apply_chat_template keeps the generated text focused on the actual answer.
- **tqdm progress** is shown while batch-generating answers for a whole dataset (answer_dataset).

7. Evaluation system

src/evaluation.py implements recall@k (k = 1, 3, 5, 10) for the team's own iteration — the official recall@k score at soutenance is computed by the moulinette's own evaluate_student_search_results, never by this module.

- A retrieved source is considered a **match** for a ground-truth source when both refer to the same file_path and their character ranges overlap with an Intersection-over-Union (IoU) of at least 5%.
- **Per-question score:** number of ground-truth sources matched, divided by the total number of ground-truth sources for that question.
- **Matching** is done by question_id between the StudentSearchResults file and the AnsweredQuestion entries of the ground-truth RagDataset; questions without ground truth are skipped.
- The report warns when a requested k exceeds the k actually used at retrieval time (e.g. recall@10 cannot be measured from a search run with k=5).

The implementation was cross-checked against the real moulinette binary (evaluate_student_search_results) and produces numerically identical results, confirming the matching logic reproduces the official scoring exactly.

8. Performance analysis

All numbers below were measured on the real vLLM 0.10.1 corpus (1969 indexable files, 18733 chunks).

Metric	Result	Requirement	Status
Indexing time	3.4 s	< 5 min	PASS
Throughput (200 questions)	0.38 s	< 90 s	PASS
Recall@5 — docs questions	84.0%	≥ 80%	PASS
Recall@5 — code questions	70.7%	≥ 50%	PASS

Recall was measured with the project's own `evaluate` command — and independently confirmed with the real `moulinette`, which reports identical numbers — against the provided ground-truth datasets (`dataset_docs_public.json`, 100 questions, and `dataset_code_public.json`, 99 questions), retrieving with `k=10`:

Question set	Recall@1	Recall@3	Recall@5	Recall@10
docs	63.0%	77.0%	84.0%	87.0%
code	39.4%	66.7%	70.7%	79.8%

Both indexing and retrieval are far faster than required — BM25 over a corpus this size is cheap. Recall clears both thresholds with a comfortable margin today, but this wasn't always the case: before adding identifier expansion (§5), code recall@5 was only 50.5% — just barely above the 50% bar, and one bad corpus variation away from failing.

9. Command-line interface

Every command is exposed through a Python Fire CLI, invoked as `uv run python -m src <command> [options]`, and degrades gracefully on bad input (empty query, `k=0`, `k` larger than the corpus, a missing index or dataset file, malformed JSON, a missing model, a dataset with no ground truth) instead of raising an unhandled traceback.

```
# Build the BM25 index from the raw corpus
uv run python -m src index --max_chunk_size 2000

# Search for the top-5 most relevant chunks for a question
uv run python -m src search "What is a KV cache?" -k 5

# Batch search over a dataset of questions
uv run python -m src search_dataset \
  --dataset_path data/datasets/UnansweredQuestions/dataset_docs_public.json \
  --k 10 --save_directory data/output/search_results/AnsweredQuestions

# Generate a single grounded answer
uv run python -m src answer "How does automatic prefix caching work?" -k 5

# Batch-generate answers from a StudentSearchResults file
uv run python -m src answer_dataset \
  --student_search_results_path data/output/search_results/AnsweredQuestions/dataset_docs_public.json \
  --save_directory data/output/search_results_and_answer/AnsweredQuestions

# Evaluate recall@k against ground-truth sources (own use only)
```

```
uv run python -m src evaluate \  
  --student_search_results_path data/output/search_results/AnsweredQuestions/dataset_docs_public.json \  
  --dataset_path data/datasets/AnsweredQuestions/dataset_docs_public.json
```

10. Design decisions

- **BM25 over TF-IDF / embeddings.** Chosen as the mandatory baseline retrieval method for its strong out-of-the-box performance on heterogeneous corpora without needing a trained embedding model or GPU.
- **AST-based chunking for Python.** Keeps functions and classes whole instead of chunking on a fixed window, which matters for code questions where the answer is usually a whole function's worth of context.
- **Identifier expansion, additive not destructive.** Appending split subwords to both corpus and query text before tokenization (rather than replacing the original text) preserves exact-identifier matching while adding natural-language matching on top — a strictly additive change that measurably helped code recall without needing an embedding model.
- **Re-reading chunk text from disk at generation/evaluation time.** MinimalSource only stores a file path and a character range. Rather than duplicating content into the index, the actual text is read back from `data/raw/` when needed — smaller index, single source of truth for content. Trade-off: `data/raw/` must remain on disk for `answer`, `answer_dataset` and `evaluate` to work.
- **CPU-only PyTorch, pinned explicitly.** `torch` resolves to a CUDA build by default, downloading gigabytes of `nvidia-*` packages that Qwen3-0.6B doesn't need and that can fill a constrained disk. Pinning the CPU wheel index in `pyproject.toml` keeps `uv sync` fast, small, and reproducible on any grading machine, GPU or not.
- **`enable_thinking=False` for Qwen3.** Keeps the generated JSON focused on the actual answer instead of leaking chain-of-thought noise into `MinimalAnswer.answer`.
- **Every CLI command fails gracefully.** Required by the subject, and generally useful when iterating by hand.

11. Challenges faced

- **bm25s edge cases.** The library raises an unhelpful `ValueError` when indexing an empty corpus, and again when retrieving with `k` larger than the corpus size. Both are guarded explicitly with clear error messages instead of letting the library's raw exception surface.
- **transformers' chat template return type.** Calling `tokenizer.apply_chat_template(..., return_tensors="pt")` for Qwen3 returns a `BatchEncoding` (a dict with `input_ids/attention_mask`), not a bare tensor — passing it straight to `model.generate()` as a positional argument silently mis-parses it. Fixed by requesting the templated prompt as plain text (`tokenize=False`), tokenizing it explicitly, then unpacking into `model.generate(**encoded, ...)`.
- **torch's default CUDA build filling the disk.** On the development machine (an 18GB disk, often nearly full), a plain `torch` dependency resolved to a CUDA build pulling in ~2GB of `nvidia-*/cuda-*` packages and repeatedly exhausted available disk space — including once after the fix had already been applied, when an unrelated `uv run` invocation silently re-triggered a `resync` that reintroduced the CUDA build. Solved by pinning an explicit CPU wheel index for `torch` and re-verifying `torch.version.cuda` is `None` after every dependency change.

- **Recall@5 margin on code questions.** An initial version passed the code threshold with only +0.5 point of margin (50.5% vs. 50%). Digging into bm25s's tokenizer showed why: it tokenizes on word boundaries only, so identifiers like `get_kv_cache_size` never matched natural-language questions about a “KV cache”. Adding identifier expansion raised code recall@5 to 70.7% without touching docs recall (84.0%, actually up slightly from 82.0%).

12. AI usage

This project was built with the assistance of Claude Code (Anthropic), used throughout the implementation as a pair-programming tool: designing and writing the AST-based Python chunker and the paragraph-based Markdown chunker, working out the exact bm25s API for persisting and reloading per-chunk metadata, implementing the retrieval, answer-generation and evaluation modules, diagnosing the identifier-tokenization gap behind the code-recall margin and implementing the fix, debugging integration issues (bm25s edge cases, the transformers BatchEncoding gotcha, the torch/CUDA dependency regression), running the CLI end-to-end against the real vLLM corpus and the official moulinette to verify behavior, and drafting the README and this report. All generated code was reviewed, tested against the real corpus, and adjusted before being kept.

End of report — generated from the project's TODO.md, README.md and CLAUDE.md, and the recall figures measured with both the project's own evaluate command and the official moulinette.